

State management

COMPLETE LEARNING GUIDE

What You'll Learn

As an app grows past a handful of components, deciding *where* a piece of state should live becomes one of the highest-leverage architectural decisions in frontend development. This guide walks through the full spectrum: state that's genuinely local to one component, state lifted to a shared ancestor, React's built-in Context API for avoiding deep prop drilling, and dedicated state libraries (Redux and lighter alternatives like Zustand) for large, complex applications. It also covers a distinction that trips up almost every learner moving into real-world apps: server state (data that lives on a backend and is just cached on the client) is fundamentally different from client UI state (a modal being open, a selected tab), and treating them the same way causes both unnecessary complexity and subtle bugs.

1. The State Locality Spectrum

The default, correct choice for any new piece of state is to keep it as local as possible — inside the component that actually uses it, via `useState`. Only move state further out (lifting it to a parent, or eventually to global state) once you have a concrete reason: multiple components need to read or write the same value and they aren't in a simple parent-child relationship. A common mistake is prematurely reaching for a global store 'just in case,' which adds indirection and boilerplate to state that will only ever be used by one component and its direct children.

2. React Context API

Context lets a value be read by any descendant of a `Provider` without passing it down as a prop through every intermediate layer. A Context is created with `createContext()`, populated with a value via `<MyContext.Provider value={...}>`, and read anywhere below it with `useContext(MyContext)`. Context is well-suited to relatively static, infrequently-changing data — theme, current authenticated user, locale — because every component consuming that context re-renders whenever the Provider's value changes, regardless of which specific field a given consumer actually reads. Splitting frequently-changing state into its own smaller, dedicated Context (rather than one large context object holding everything) limits the blast radius of each update.

JSX

```
const ThemeContext = createContext("light");

function App() {
  const [theme, setTheme] = useState("light");
  return (
    <ThemeContext.Provider value={theme}>
      <Toolbar onToggle={() => setTheme((t) => (t === "light" ? "dark" :
"light"))}></ThemeContext.Provider>
    );
  );
}

function Toolbar({ onToggle }) {
  const theme = useContext(ThemeContext); // no prop drilling needed
  return <button onClick={onToggle}>Current theme: {theme}</button>;
}
```

3. Redux Fundamentals

Redux centralizes all application state in a single store and enforces that the only way to change it is by dispatching a plain-object action, which a pure reducer function uses to compute the next state from the current state — never mutating it. This unidirectional flow (dispatch action !' reducer computes new state !' UI re-renders from new state) makes state changes predictable and traceable, which is why Redux DevTools can replay every action that ever happened. Modern Redux (via Redux Toolkit) drastically cuts the historical boilerplate by generating action creators and allowing reducers to be written with 'mutating' syntax internally (backed by Immer, which produces an immutable update under the hood).

JAVASCRIPT

```
import { createSlice, configureStore } from "@reduxjs/toolkit";

const counterSlice = createSlice({
  name: "counter",
  initialState: { value: 0 },
  reducers: {
    incremented: (state) => { state.value += 1; }, // Immer makes this safe
    decremented: (state) => { state.value -= 1; },
  },
});

export const { incremented, decremented } = counterSlice.actions;
export const store = configureStore({ reducer: { counter:
counterSlice.reducer } });
```

4. Zustand as a Lightweight Alternative

Zustand takes a much smaller-footprint approach: a store is just a hook created with `create()`, holding state and the functions that update it, with no actions, reducers, or Provider wrapper required. Components subscribe only to the specific slice of state they select, so — unlike naive Context usage — a component reading `store.count` doesn't re-render when `store.username` changes. This makes Zustand a popular middle ground for apps that have outgrown plain `useState/Context` but don't want Redux's ceremony.

JAVASCRIPT

```
import { create } from "zustand";

const useCounterStore = create((set) => ({
  count: 0,
  increment: () => set((state) => ({ count: state.count + 1 })),
}));

function Counter() {
  const count = useCounterStore((state) => state.count); // only re-renders on count change
  const increment = useCounterStore((state) => state.increment);
  return <button onClick={increment}>{count}</button>;
}
```

5. Server State vs. Client State

Client state (a form's current input, whether a dropdown is open) is owned entirely by the frontend. Server state (a list of students fetched from an API) is fundamentally different: it's owned by the backend, can go stale, can fail to load, may need refetching, and is often needed by multiple unrelated components. Managing server state with the same tools as client state (manually `useState` + `useEffect` fetching everywhere) reproduces a lot of hand-rolled logic — loading flags, error handling, cache invalidation, avoiding duplicate requests — that dedicated server-state libraries like TanStack Query (React Query) or SWR already solve, including automatic caching, background refetching, and request deduplication.

6. Avoiding Unnecessary Re-renders

A component re-renders when its own state changes, when its parent re-renders, or when a Context it consumes changes — not only when 'its own data' changes, which surprises many learners. `React.memo` prevents a function component from re-rendering when its props haven't changed (by shallow comparison), but it's easily defeated by passing a new inline object, array, or function as a prop on every parent render, since those are never shallow-equal to the previous render's value. `useMemo` and `useCallback` exist specifically to preserve stable references for values and functions across renders so that memoized children can actually skip re-rendering.

- `React.memo` only helps if the props passed to the component are actually stable between renders — verify with a profiler before assuming it worked.
- Splitting a large context value into multiple smaller contexts limits how many consumers re-render on any single change.

7. Choosing the Right Tool

There's no single correct answer — it's a decision based on scope and change frequency. Local component state should be the default. Lifted state covers small, closely related component groups. Context suits infrequently-changing, broadly-needed values like theme or auth. A dedicated store (Redux/Zustand) earns its cost once genuinely complex, cross-cutting client state exists (e.g., a multi-step wizard's state needed across a dozen unrelated routes). And any data that originates from a server belongs in a server-state library, not hand-

rolled fetch-and-useState logic, regardless of which client-state tool the rest of the app uses.

Common Pitfalls

- Reaching for Redux or global state on day one for state that only one component and its direct children will ever need.
- Putting frequently-changing values (like mouse position or a text input) into a single large Context, causing every consumer to re-render on every keystroke.
- Mutating Redux state directly outside of Immer-backed Redux Toolkit reducers, silently breaking change detection.
- Managing server-fetched data with plain useState/useEffect everywhere instead of a server-state library, reinventing caching and loading-state logic per component.
- Passing a new inline object or arrow function as a prop every render, which defeats React.memo on the receiving component.
- Treating 'is this modal open' (client UI state) and 'what did the API return' (server state) as the same kind of state requiring the same tool.

Key Takeaways

- Default to the most local state possible; only lift or globalize state once a concrete cross-component need appears.
- Context is best for infrequently-changing, broadly-shared values — not a general-purpose replacement for a state management library.
- Redux enforces a strict, traceable unidirectional data flow; Zustand offers similar centralized state with far less boilerplate.
- Server state and client state are different problems — use a server-state library (React Query/ SWR) for anything that originates from an API.
- A component re-renders when its parent re-renders or a consumed Context changes, not only when its 'own' data changes.
- Stable references (via useMemo/useCallback) are what make React.memo actually effective.

Learn More

React Docs — Passing Data Deeply with Context

<https://react.dev/learn/passing-data-deeply-with-context>

Redux Toolkit — Quick Start

<https://redux-toolkit.js.org/tutorials/quick-start>

Zustand — Getting Started

<https://zustand.docs.pmnd.rs/getting-started/introduction>